

1. Print K largest(or smallest) elements in an array

Given an array `arr[]` of size `N`, the task is to printing `K` largest elements in an array.

```
// C++ code for K largest elements in an array
#include <bits/stdc++.h>
using namespace std;
```

```
void kLargest(int arr[], int n, int k)
{
    // Sort the given array arr in reverse order.
    sort(arr, arr + n, greater<int>());

    // Print the first kth largest elements
    for (int i = 0; i < k; i++)
        cout << arr[i] << " ";
}
```

```
// Driver code
int main()
{
    int arr[] = { 1, 23, 12, 9, 30, 2, 50 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 3;
    kLargest(arr, n, k);
}
```

2. Check Equal Arrays

Given two arrays `arr1` and `arr2` of equal size, the task is to find whether the given arrays are equal. Two arrays are said to be equal if both contain the same set of elements, arrangements (or permutations) of elements may be different though.

Note: If there are repetitions, then counts of repeated elements must also be the same for two arrays to be equal.

```
class Solution {
public:
    // Function to check if two arrays are equal or not.
```

```

bool check(vector<int>& arr1, vector<int>& arr2) {
    sort(arr1.begin(),arr1.end());
    sort(arr2.begin(),arr2.end());
    if(arr1==arr2){
        return true;
    }
    return false;
}

```

3.Missing In Array

Given an array **arr** of size **n-1** that contains distinct integers in the range of 1 to n (inclusive), find the missing element. The array is a permutation of size n with one element missing. Return the missing element.

```

// User function template for C++
class Solution {
public:

    // Note that the size of the array is n-1
    int missingNumber(int n, vector<int>& arr) {

        int sum=0;
        for(int i=0;i<n-1;i++){
            sum+= arr[i];
        }
        int ans= n*(n+1)/2;
        return ans-sum;
    }
};

```

4. Count Inversions

Given an array of integers. Find the Inversion Count in the array. Two array elements **arr[i]** and **arr[j]** form an inversion if **arr[i] > arr[j]** and **i < j**.

Inversion Count: For an array, inversion count indicates how far (or close) the array is from being sorted. If the array is already sorted then the inversion count is 0.

If an array is sorted in the reverse order then the inversion count is the maximum.

```
class Solution {
public:
    // arr[]: Input Array
    // N : Size of the Array arr[]
    // Function to count inversions in the array.
    long long merge(long long arr[], int st, int mid, int end)
    {
        long long cnt=0;
        vector<long long>temp;
        int l=st,r=mid+1;
        while(l<=mid && r<=end)
        {
            if(arr[l]<=arr[r])
            {
                temp.push_back(arr[l]);
                l++;
            }
            else
            {
                temp.push_back(arr[r]);
                cnt+=(mid-l+1);
                r++;
            }
        }
        while(l<=mid)
        {
            temp.push_back(arr[l]);
            l++;
        }
        while(r<=end)
        {
            temp.push_back(arr[r]);
            r++;
        }
        for(int i=st;i<=end;i++)
        {
            arr[i]=temp[i-st];
        }
        return cnt;
    }
    long long mergesort(long long arr[], int st, int end)
```

```

{
    long long cnt=0;
    if(st>=end) return cnt;
    int mid=(st+end)/2;
    cnt+=mergesort(arr,st,mid);
    cnt+=mergesort(arr,mid+1,end);
    cnt+=merge(arr,st,mid,end);
    return cnt;
}
long long int inversionCount(long long arr[], int n) {
    // Your Code Here
    long long int ans=mergesort(arr, 0, n-1);
    return ans;
}
};

```

5. Merge Two Sorted Array

```

class Solution{
public:
    //Function to merge the arrays.
    void merge(long long arr1[], long long arr2[], int n, int m)
    {
        long long arr3[n+m];
        int i=0,j=0,k=0;
        while(i<n && j<m){
            if(arr1[i]<arr2[j]){
                arr3[k] =arr1[i];
                i++,k++;
            }
            else {
                arr3[k] = arr2[j];
                j++,k++;
            }
        }
        while(i<n){
            arr3[k] = arr1[i];
            i++,k++;
        }
        while(j<m){
            arr3[k] = arr2[j];
            j++,k++;
        }
        for(int i=0;i<n;i++) arr1[i] = arr3[i];
        for(int i=n;i<n+m;i++) arr2[i-n] =arr3[i];
    }
};

```

6. Equilibrium Index/Point

Given an array **arr** of non-negative numbers. The task is to find the first **equilibrium point** in an array. The equilibrium point in an array is an index (or position) such that the **sum** of all elements **before** that index is the **same** as the **sum** of elements **after** it.

Note: Return equilibrium point in 1-based indexing. Return -1 if no such point exists

```
class Solution {
public:
    // Function to find equilibrium point in the array.
    // arr: input array
    int equilibriumPoint(vector<long long> &arr) {
        // Your code here
        int total_sum = 0;
        int left_sum = 0;

        //calc the total sum
        for (int i = 0; i < arr.size(); i++){
            total_sum += arr[i];
        }

        int right_sum = total_sum;
        for (int i = 0; i < arr.size(); i++){
            right_sum -= arr[i];

            if (right_sum == left_sum){
                return i+1; // for base 1 index
            }
            left_sum += arr[i];
        }
        return -1;
    }
};
```

7. Median of 2 Sorted Arrays of Different Sizes

Given two sorted arrays array1 and array2 of size m and n respectively. Find the median of the two sorted arrays.

```
class Solution{
public:
double MedianOfArrays(vector<int>& array1, vector<int>& array2)
{
    vector <int> mergedArray;
    int i= 0 , j= 0;

    while(i<array1.size() && j<array2.size())
    {
        if(array1[i]<array2[j])
        {
            mergedArray.push_back(array1[i]);
            i++;
        }
        else
        {
            mergedArray.push_back(array2[j]);
            j++;
        }
    }

    while(i<array1.size())
    {
        mergedArray.push_back(array1[i++]);
    }

    while(j<array2.size())
    {
        mergedArray.push_back(array2[j++]);
    }

    //find median
    int totalsize= mergedArray.size();
    if(totalsize % 2 == 0)
    {
        return( mergedArray[totalsize/2 - 1 ] + mergedArray[totalsize/2])/2.0;
    }

    else
    {
        return mergedArray[totalsize/2];
    }
}
};
```

8.Insert Node in bst

Given a **BST** and a key **k**. If **k** is not present in the BST, Insert a new Node with a value equal to **k** into the BST. If **k** is already present in the BST, don't modify the BST. Return the root of the modified BST after inserting **k**.

class Solution

```
{
    public:
        void find(Node* root,int data,bool &pres){
            if(root==NULL){
                return;
            }
            if(root->data == data){
                pres= true;
                return;
            }
            if (root->data > data) {
                find(root->left, data, pres);
            } else {
                find(root->right, data, pres);
            }
        }
        Node* solve(Node* node,int data){
            if(node==NULL){
                Node* temp = new Node(data);
                return temp;
            }

            if(node->data > data){
                node->left = solve(node->left,data);
            }else{
                node->right = solve(node->right,data);
            }
            return node;
        }
        Node* insert(Node* node, int data) {
            bool pres = false;
            find(node,data,pres);

            if(pres){
                return node;
            }else{
                return solve(node,data);
            }
        }
};
```

9. Triangle Path Sum

Given a **triangle** array, return the **minimum path sum** to reach from **top** to **bottom**.

For each step, you may move to an adjacent number of the row below. More formally, if you are on index i on the **current row**, you may move to either index i or index $i + 1$ on the **next row**.

```
class Solution {
public:
    int minPath(int i, int j, int n, vector<vector<int>>&tri, vector<vector<int>>&dp){
        if(j>i || j>=n)
            return 0;
        if(dp[i][j]!=-1){
            return dp[i][j];
        }
        dp[i][j]=tri[i][j]+min(minPath(i+1,j,n,tri,dp),minPath(i+1,j+1,n,tri,dp));
        return dp[i][j];
    }
    int minimizeSum(int n, vector<vector<int>>& triangle) {
        // Code here
        vector<vector<int>>dp(n,vector<int>(n,-1));
        for(int i=0;i<n;i++){
            dp[n-1][i]=triangle[n-1][i];
        }

        return minPath(0,0,n,triangle,dp);
    }
};
```

10.Coin Change

Given an integer array coins[] of size N representing different denominations of currency and an integer sum, find the number of ways you can make sum by using different combinations from coins[].

Note: Assume that you have an infinite supply of each type of coin. And you can use any coin as many times as you want.

```
class Solution {
public:
    long long int count(int coins[], int N, int sum) {

        // code here.
        vector<vector<long long>> memo(N+1, vector<long long>(sum+1, -1));

        return helper(coins, N, sum, memo);
    }

    long long helper(int coins[], int N, int sum, vector<vector<long long>> & memo)
    {
        if(sum == 0) return 1;
    }
```



```

        if (N == 0) return 0;

        if (memo[N][sum] != -1) return memo[N][sum];

        if (coins[N-1] <= sum)
        {
            memo[N][sum] = helper(coins, N, sum - coins[N-1], memo) + helper(coins, N-1,
sum, memo);
        }
        else{
            memo[N][sum] = helper(coins, N-1, sum, memo);
        }

        return memo[N][sum];
    }
};

```

11. Binary Search

Given a sorted array arr and an integer k, find the position(0-based indexing) at which k is present in the array using binary search.

```

class Solution {
public:

    int binaryprbl(vector<int> &arr, int s, int e, int k) {

        if (s > e)
            return -1;

        int mid = s + (e - s) / 2;

        if (arr[mid] == k) {
            return mid;
        }
        if (arr[mid] < k) {
            return binaryprbl(arr, mid + 1, e, k);
        } else {
            return binaryprbl(arr, s, mid - 1, k);
        }
    }

    int binarysearch(vector<int> &arr, int k) {
        return binaryprbl(arr, 0, arr.size() - 1, k);
    }
};

```

11. Floyd Warshall

The problem is to find the shortest distances between every pair of vertices in a given edge-weighted directed graph. The graph is represented as an adjacency matrix of size $n \times n$. $\text{Matrix}[i][j]$ denotes the weight of the edge from i to j . If $\text{Matrix}[i][j] = -1$, it means

there is no edge from i to j.

Note : Modify the distances for every pair in-place.

```
class Solution {
public:
    void shortest_distance(vector<vector<int>>&matrix){
        // Code here
        int n=matrix.size();

        for(int i=0;i<n;i++)
        { matrix[i][i]=0;
            for(int j=0;j<n;j++)
            {
                if(matrix[i][j]==-1)matrix[i][j]=1e9;
            }
        }

        for(int k=0;k<n;k++)
        {
            for(int i=0;i<n;i++)
            {
                for(int j=0;j<n;j++)
                {

                    matrix[i][j]=min(matrix[i][j],matrix[i][k]+matrix[k][j]);

                }
            }
        }

        for(int i=0;i<n;i++)
        {

            for(int j=0;j<n;j++)
            {
                if(matrix[i][j]==1e9)matrix[i][j]=-1;
            }
        }

    }
};
```

12 .Smallest subarray with sum greater than x

Given a number x and an array of integers arr, find the smallest subarray with sum greater than the given value. If such a subarray do not exist return 0 in that case.

```

class Solution {
public:
    int smallestSubWithSum(int x, vector<int>& arr) {
        // 15 1 2 3 9 1 1 1 14 9
        int sum=0;

        int i=0;
        int n=arr.size();
        int mini=INT_MAX;
        for(int j=0;j<n;j++){
            sum+=arr[j];
            while(sum>x){
                sum=sum-arr[i];
                mini=min(mini,j-i+1);
                i++;
            }
        }
        if(mini==INT_MAX){
            return 0;
        }
        return mini;
    }
};

```